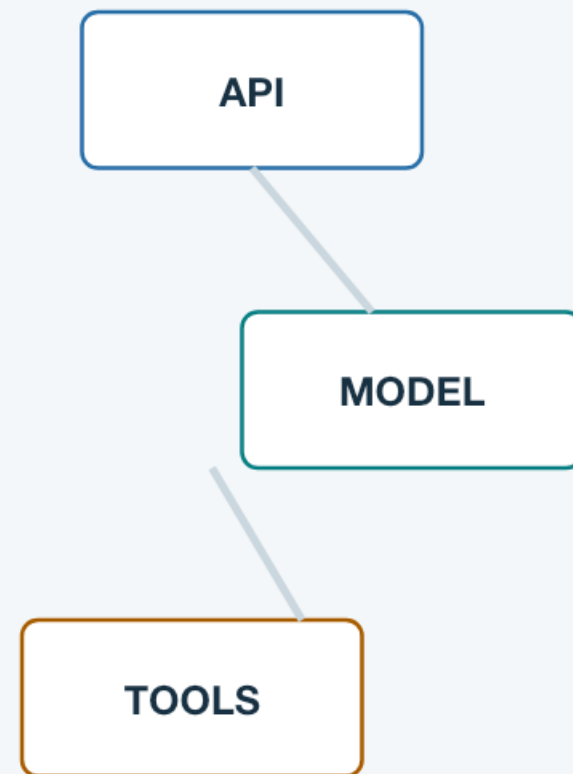


When AI Joins the Request Path

Securing API-backed AI systems

Jayakumar Ramalingam

Conf42 DevSecOps 2026



One request can cross six trust boundaries



Every hop changes the data, authority, or failure mode

Treat the full path as one security boundary map

Classic API controls remain necessary

1

Identity

Strong authentication and token validation

2

Authorization

Object and function checks on every resource

3

Resource limits

Payload, rate, concurrency, and cost budgets

4

Inventory

Known endpoints, versions, models, and dependencies

AI adds ambiguity inside a valid request

Probabilistic output

A valid input can produce an unsafe or malformed answer

Dynamic context

Retrieved text can contain instructions or sensitive data

Automated action

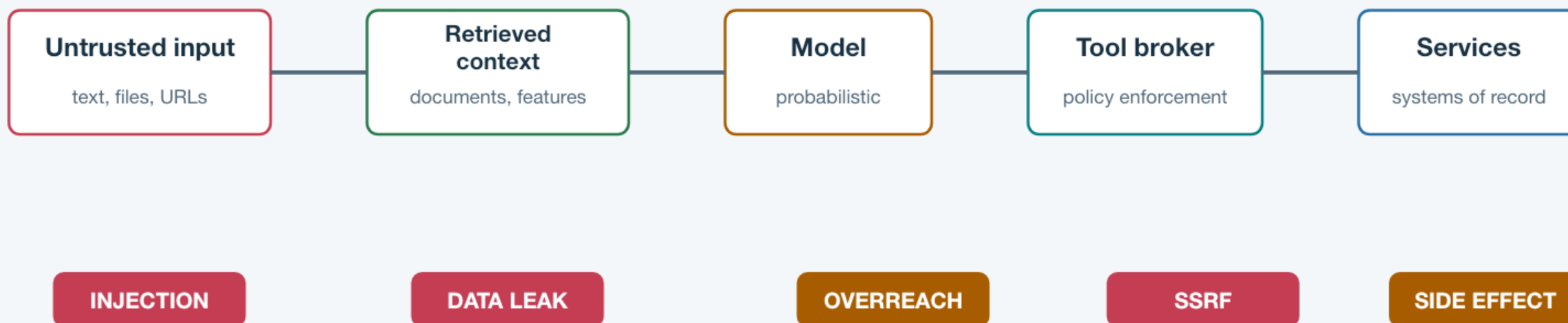
A model suggestion can become a tool call with real impact

Silent fallback

A timeout may bypass policy or expose a weaker path

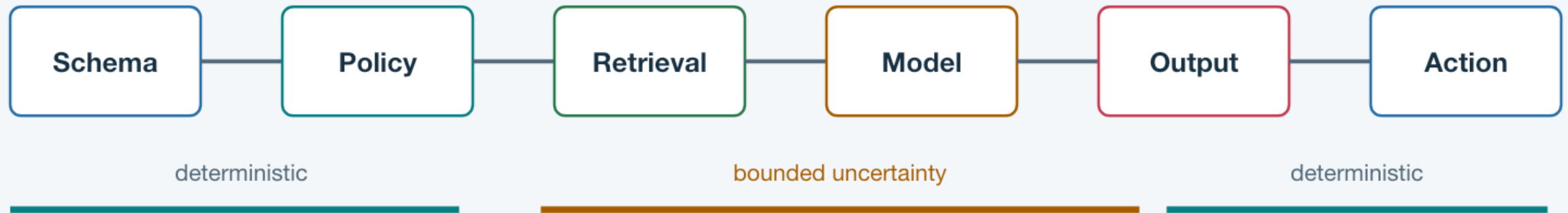
A 200 response only proves transport success

Threat modeling follows data and authority



Lethal trifecta: private data, untrusted content, and an outbound path

Validation belongs before and after inference



Keep policy decisions outside the model

Authorization is checked at every hop

API

Can this caller use this endpoint?

Object

Can this caller access this record or tenant?

Context

May these documents enter this request?

Tool

May this user invoke this operation now?

Service

Does the downstream system agree?

Confused deputy: broad agent authority acting for a user with narrow rights

Retrieved context is untrusted input

BEFORE RETRIEVAL

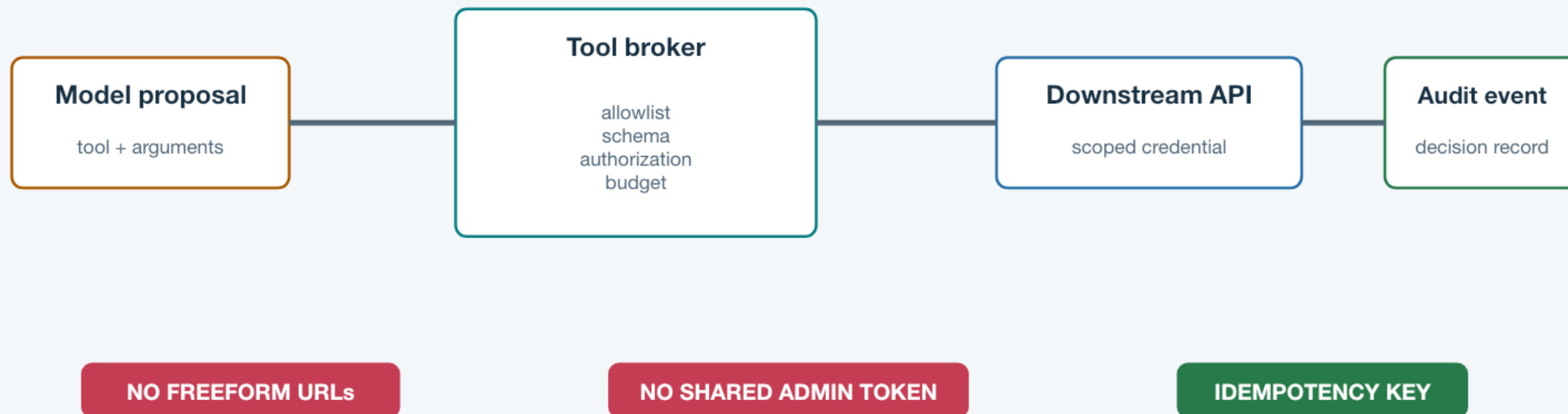
- Filter by tenant and entitlement
- Allow approved sources and URI schemes
- Limit document count, size, and age
- Preserve source and chunk provenance

AFTER RETRIEVAL

- Separate data from instructions
- Scan for secrets and restricted fields
- Treat citations as evidence, not authority
- Recheck policy before any action

Index write access is model privilege

Tool calls use a narrow broker



Budgets limit both abuse and accidents

REQUEST

bytes + tokens

1

RETRIEVAL

documents + fan-out

2

MODEL

latency + spend

3

TOOLS

calls + side effects

4

Enforce one end-to-end request budget

Request counts miss cost variance. One request can cost 1,000× another.

Logs need evidence without raw secrets

RECORD

- ✓ request and trace IDs
- ✓ caller and tenant IDs
- ✓ policy decisions
- ✓ model and prompt versions
- ✓ retrieval source IDs
- ✓ tool name and outcome
- ✓ fallback reason

REDACT OR OMIT

- ✗ access tokens
- ✗ raw secrets
- ✗ full private documents
- ✗ unredacted prompts by default
- ✗ sensitive tool responses

Fallbacks preserve policy and reduce capability

FULL PATH

retrieval + model + approved tools

LIMITED PATH

model without tools or fresh context

DETERMINISTIC

cached or rules-based answer

SAFE STOP

explicitly unavailable

Observe decisions, not only infrastructure

DENY RATE

policy blocks by reason

CONTEXT

cross-tenant or empty retrieval

OUTPUT

validation and redaction failures

TOOLS

calls, retries, and denied actions

FALLBACK

rate and trigger

COST

tokens and fan-out per request

The delivery pipeline tests policy and behavior



Version models, prompts, policies, retrieval configs, and tool contracts together

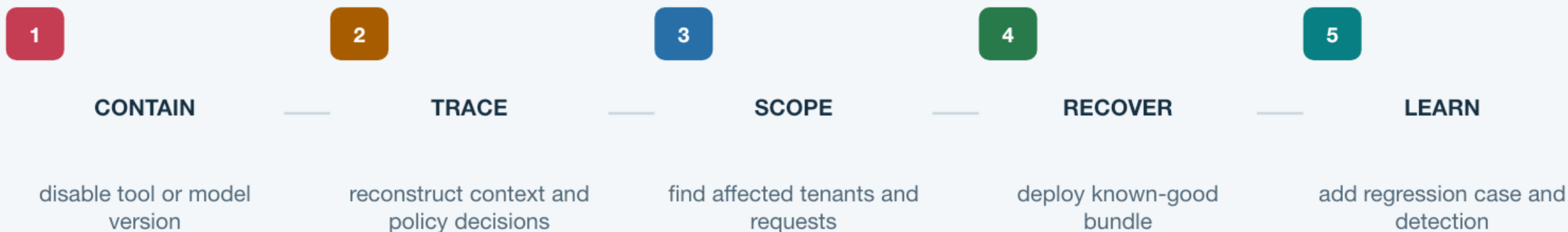
Roll back the decision system as a unit

Test the paths that normal demos avoid

- | | | |
|---|----------------------------|---|
| 1 | Boundary tests | oversized payloads, Unicode, malformed structure |
| 2 | Authorization tests | object, tenant, context, tool, downstream |
| 3 | Adversarial tests | prompt injection, secret extraction, SSRF |
| 4 | Resilience tests | timeouts, partial results, retries, duplicate actions |
| 5 | Regression tests | same corpus across model and policy versions |

Model tests are distributions: repeat runs, gate on pass rate, pin model versions in CI

Incident response needs replayable evidence



A model rollback without prompt, policy, and retrieval versions is incomplete

A practical production checklist

- | | |
|--|---|
| ✓ Map data and authority across the request path | ✓ Validate before inference and before action |
| ✓ Enforce authorization at API, context, tool, and service | ✓ Give tools narrow schemas and scoped credentials |
| ✓ Set one end-to-end budget | ✓ Log decisions without copying sensitive payloads |
| ✓ Design a capability-reducing fallback ladder | ✓ Test and observe model, policy, context, and tools together |

Secure the decision chain

The API contract now includes context, inference, policy, tools, and fallback behavior

Thank you

Jayakumar Ramalingam